POLITECNICO
MILANO 1863

Ingegneria del software

Design pattern creazionali

Factory method

Motivazioni

- Ipotizziamo che un sistema software gestisca la creazione e l'editing di diversi tipi di documenti (e.g., documenti di testo, fogli di calcolo)
- Nel sistema sono definite due principali classi astratte *Application* e *Document.*
- Per ciascuna tipologia di applicazione vengono definite relative sottoclassi (e.g., *TextApplication* e *TextDocument*)
- La classe astratta *Application* ha il compito di gestire documenti, creandoli o aprendoli quando necessario

Motivazioni

- **Problema:** *Application* non può prevedere quale tipo di documento l'utente vuole creare
- Solamente le sottoclassi di Application conoscono il tipo da istanziare
- Anti-pattern: utilizzare il tipo di una sottoclasse in una superclasse

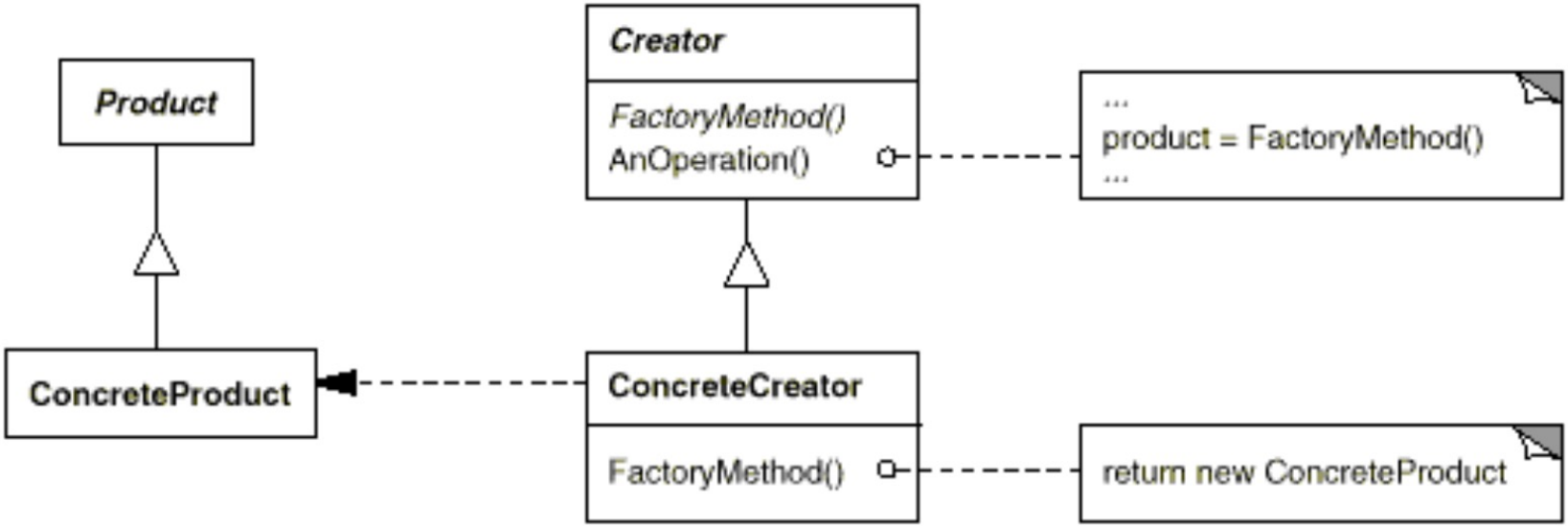
Anti-pattern

```
abstract class Application {
        …
        public Document newDocument(){
                …
                Document d;
                if (this instanceof TextApplication)
                        d = new TextDocument()
                else if (this instanceof CalcApplication)
                        d = new CalcDocument()
                …
                return d;
        }
};
```

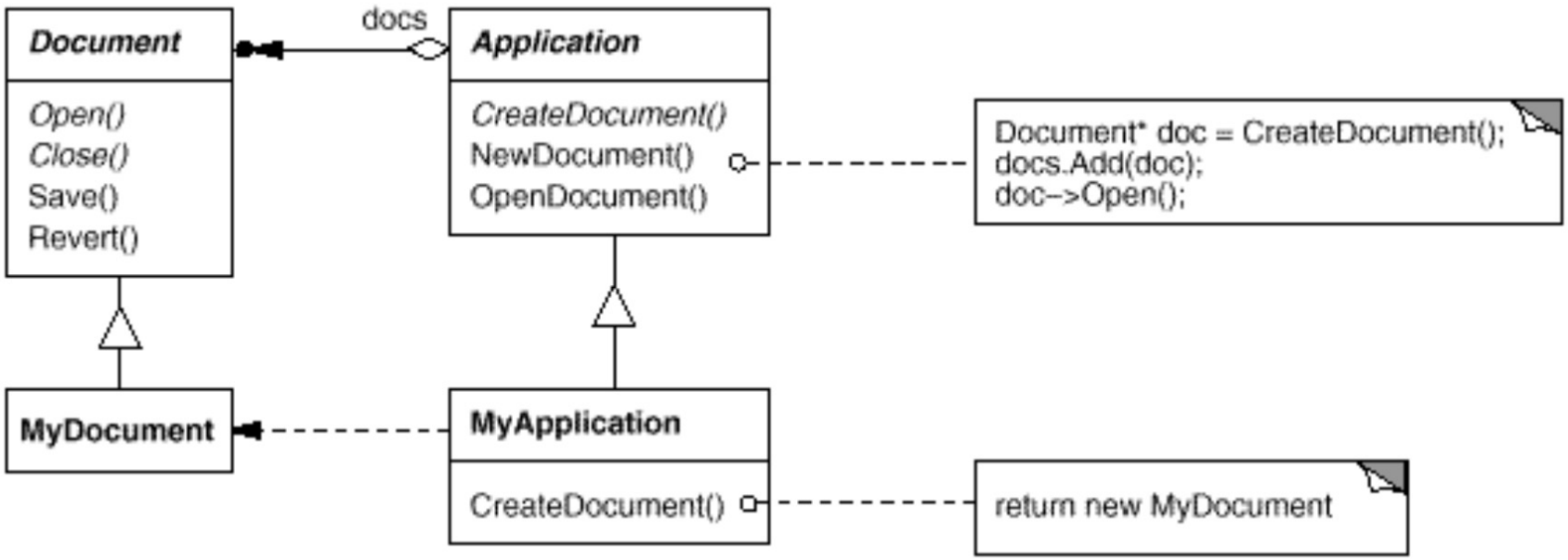
Obbiettivi

- Definire nella classe astratta un metodo astratto “factory” per la creazione di oggetti
- Le sottoclassi conoscono il tipo da instanziare
- Il pattern delega la creazione di oggetti alle sottoclassi
- La super classe non contiene dipendenze con tipi specifici

Struttura


Product
Creator
FactoryMethod()
AnOperation()
...
product = FactoryMethod()
...
ConcreteProduct
ConcreteCreator
FactoryMethod()
return new ConcreteProduct

Esempio


Document
Open()
Close()
Save()
Revert()
docs
Application
CreateDocument()
NewDocument()
OpenDocument()
Document* doc = CreateDocument();
docs.Add(doc);
doc->Open();
MyDocument
MyApplication
CreateDocument()
return new MyDocument

Esempio

```
abstract class Application {
        …
        public Document newDocument(){
                …
                Document d = createDocument();
                …
                return d;
        }

        public abstract Document createDocument();
};
```

Esempio

```
class TextApplication extends Application {
      public Document createDocument() {
            return new TextDocument();
      }
};
```